

1. Script to Find World-Writable Files

Answer:

A world-writable file means **others (everyone) have write permission (w)**.

In Linux, this is dangerous because anyone can modify the file.

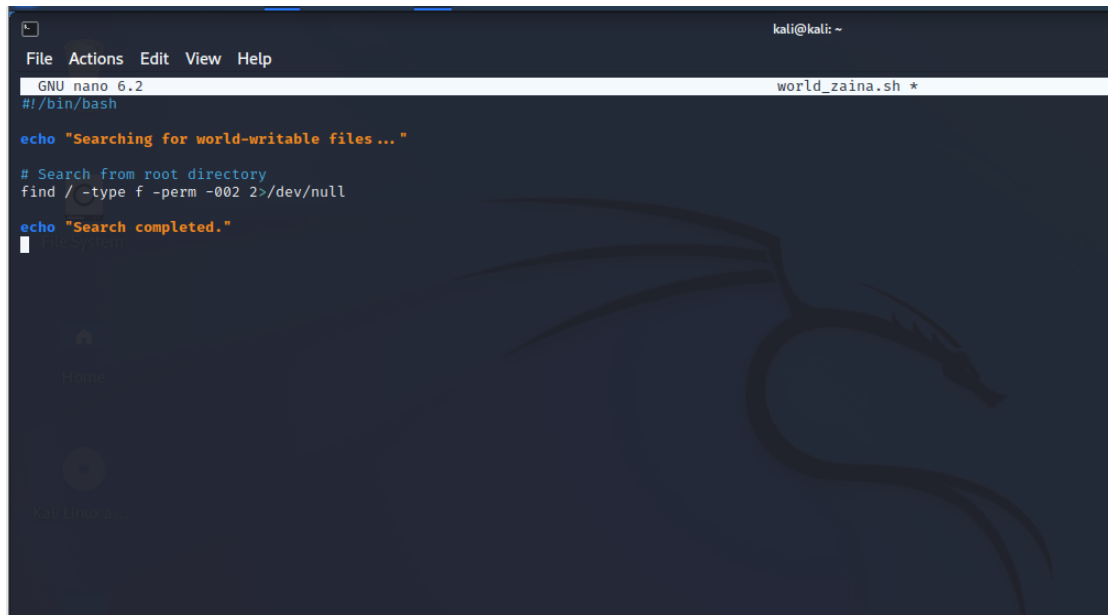
Bash Script:

```
#!/bin/bash

echo "Searching for world-writable files..."

# Search from root directory
find / -type f -perm -002 2>/dev/null

echo "Search completed."
```

A screenshot of a terminal window running the GNU nano 6.2 editor. The terminal shows the execution of a bash script named world_zaina.sh. The script's content is displayed on the screen: a shebang line, an echo statement, a comment, a find command, and another echo statement. The terminal background features a Kali Linux dragon logo. The prompt 'kali@kali: ~' is visible in the top right corner.

```
kali@kali: ~
File Actions Edit View Help
GNU nano 6.2 world_zaina.sh *
#!/bin/bash

echo "Searching for world-writable files ..."

# Search from root directory
find / -type f -perm -002 2>/dev/null

echo "Search completed."
█
```

```
File Actions Edit View Help
(kali@kali)-[~]
$ nano world_zaina.sh
(kali@kali)-[~]
$ chmod +x world_zaina.sh
(kali@kali)-[~]
$ ./world_zaina
zsh: no such file or directory: ./world_zaina
(kali@kali)-[~]
$ ./world_zaina.sh
Searching for world-writable files...
/sys/kernel/security/apparmor/.remove
/sys/kernel/security/apparmor/.replace
/sys/kernel/security/apparmor/.load
/sys/kernel/security/apparmor/.access
/sys/kernel/security/tomoyo/self_domain

```

```
kali@kali: ~
File Actions Edit View Help
/proc/2965/attr/current
/proc/2965/attr/exec
/proc/2965/attr/fscreate
/proc/2965/attr/keycreate
/proc/2965/attr/sockcreate
/proc/2965/attr/apparmor/current
/proc/2965/attr/apparmor/exec
/proc/2965/timerslack_ns
/proc/3560/task/3560/attr/current
/proc/3560/task/3560/attr/exec
/proc/3560/task/3560/attr/fscreate
/proc/3560/task/3560/attr/keycreate
/proc/3560/task/3560/attr/sockcreate
/proc/3560/task/3560/attr/apparmor/current
/proc/3560/task/3560/attr/apparmor/exec
/proc/3560/attr/current
/proc/3560/attr/exec
/proc/3560/attr/fscreate
/proc/3560/attr/keycreate
/proc/3560/attr/sockcreate
/proc/3560/attr/apparmor/current
/proc/3560/attr/apparmor/exec
/proc/3560/timerslack_ns
/proc/3561/task/3561/attr/current
/proc/3561/task/3561/attr/exec
/proc/3561/task/3561/attr/fscreate
/proc/3561/task/3561/attr/keycreate
/proc/3561/task/3561/attr/sockcreate
/proc/3561/task/3561/attr/apparmor/current
/proc/3561/task/3561/attr/apparmor/exec
/proc/3561/attr/current
/proc/3561/attr/exec
/proc/3561/attr/fscreate
/proc/3561/attr/keycreate
/proc/3561/attr/sockcreate
/proc/3561/attr/apparmor/current
/proc/3561/attr/apparmor/exec
/proc/3561/timerslack_ns
Search completed.
(kali@kali)-[~]
$
```

Explanation

- `find /` → starts searching from the root directory
- `-type f` → looks for files only
- `-perm -002` → finds files where **others have write permission**
- `2>/dev/null` → hides permission denied errors

2. How Windows NTFS Implements DAC

Answer:

Windows NTFS uses **Discretionary Access Control (DAC)** through something called **Access Control Lists (ACLs)**.

Explanation:

- Every file or folder in NTFS has a **security descriptor**
- This descriptor contains an **Access Control List (ACL)**
- The ACL includes multiple **Access Control Entries (ACEs)**

How it works:

- Each ACE defines:
 - A user or group
 - Their permissions (read, write, execute, full control)

Example:

- User A → Read only
- User B → Full control

Key Idea:

Just like Linux DAC:

- The **owner of the file decides who can access it**
- Permissions can be **granted or removed anytime**

Features of NTFS DAC:

- Supports users and groups
- Fine-grained permissions (more detailed than Linux)
- Permissions inheritance from parent folders

Limitation:

- If permissions are set incorrectly → security risk
- Similar to Linux chmod 777 problem